

Contents

The Socket Exchange — System Implementation and Experimental Analysis	2
1. System Architecture & Implementation Decisions	2
1.1 Concurrency and I/O Multiplexing Architecture	2
1.2 Architectural Rationale	2
1.3 Key Design Details	2
2. Experiment 1 — Listening and Connected Sockets	3
2.1 Technical Analysis	3
2.2 Investigation Methodology & Tools	3
2.3 Empirical Evidence	3
3. Experiment 2 — Observing TCP Connection States	4
3.1 Technical Analysis	4
3.2 Investigation Methodology & Tools	4
3.3 Empirical Evidence	5
4. Experiment 3 — TCP as a Byte Stream	6
4.1 Technical Analysis	6
4.2 Investigation Methodology & Tools	6
4.3 Empirical Evidence	7
5. Experiment 4 — One Client Should Not Stall the Others	8
5.1 Technical Analysis	8
5.2 Comparative Investigation (A/B Control)	8
5.3 Empirical Evidence	8
6. Experiment 5 — Multiple Clients and I/O Multiplexing	9
6.1 Technical Analysis	9
6.2 Investigation Methodology & Tools	9
6.3 Empirical Evidence	10
7. Experiment 6 — FIN vs. RST: Orderly and Abrupt Termination	12
7.1 Technical Analysis	12
7.2 Investigation Methodology & Tools	12
7.3 Empirical Evidence	13
8. Experiment 7 — Backpressure and the Slow Receiver	14
8.1 Technical Analysis & Theoretical Capacity	14
8.2 Initial Arithmetic vs. FreeBSD Loopback Behavior	14
8.3 Platform Investigation: FreeBSD Loopback Flow Control	16
8.4 High-Volume Verification: Run E	17
8.5 Matching Engine Decoupling Proof	20
9. Experiment 8 — Unexpected Client Disconnection	21
9.1 Technical Analysis	21
9.2 Investigation Methodology & Tools	21
9.3 Empirical Evidence	21
9.4 Supplementary Proof: Resting Order Survival Post-Disconnect	23
10. System Concurrency & Protocol Verification	23
10.1 Handout §4.3 Concurrency Verification (≥ 10 Clients)	23
10.2 Outbound and Inbound Message Coalescing	23
10.3 Protocol Error Vocabulary & Errno Matrix	23
11. Codebase Structure & Reproduction Commands	24
11.1 Source Code Architecture	24
11.2 Reproduction Commands	24
12. Edge-Case Verification	24

The Socket Exchange — System Implementation and Experimental Analysis

COL334: Computer Networks (Assignment 2)

Authors: Amar Sinha (2024CS10388), Ritwik Sehrawat (2024AM10224)

Implementation: Python 3 (`socket` and `select.kqueue` system interfaces)

Environment: FreeBSD 14.4 / 15.1-RELEASE VM, loopback interface (lo0)

1. System Architecture & Implementation Decisions

1.1 Concurrency and I/O Multiplexing Architecture

The Exchange Server is implemented as a single-threaded event loop built directly on FreeBSD's kernel event notification facility, `select.kqueue()`. High-level abstractions such as `asyncio` (prohibited by §4.1.2) and the Python `selectors` module (which obscures underlying system calls) were strictly avoided.

Every socket operation—`socket()`, `bind()`, `listen()`, `accept()`, `send()`, `recv()`, `close()`, and `shutdown()`—is invoked directly through standard low-level system interfaces.

The architecture enforces strict separation between transport management and application state:

- `server.py`: Owns all file descriptors, network buffers, and event loop scheduling.
- `framing.py`, `protocol.py`, `engine.py`: Handle stream framing, message parsing, and order matching without importing or referencing `socket`.

1.2 Architectural Rationale

1. Keep networking out of the matching engine:

The matching engine never accesses a client socket. `engine.handle(sid, parsed_msg)` receives a session identifier and returns a list of `(target_sid, message_bytes)` pairs. Only `server.flush()` writes those bytes to the network. A slow socket can therefore delay its own output without making the matching code wait on a network operation.

2. Use one event loop instead of one thread per client:

Most connections are idle, and order matching consists of short dictionary and FIFO-dequeue operations. `kqueue()` lets one thread sleep until a descriptor is ready and then process only ready sockets. A thread-per-client design would add context switches, locks, and GIL coordination without helping this small CPU workload.

3. Keep shared state simple:

The event loop calls the matching engine serially, so two handlers never modify the order book at the same time. This removes the need for locks and makes the order in which requests change shared state predictable.

1.3 Key Design Details

• Non-blocking writes:

Every connection has a `bytearray` named `wbuf` and an offset marking the bytes already sent. New messages are appended and sent immediately when possible. If `send()` raises `BlockingIOError`, the remaining bytes stay in the buffer and a `KQ_FILTER_WRITE` watch is enabled for that socket. The watch is disabled as soon as the buffer empties, because leaving a level-triggered write watch enabled would make the event loop spin.

• Close sockets after the event batch:

The server does not close a descriptor while it is still walking through a batch returned by `kevent()`. Otherwise the operating system could reuse that descriptor for a newly accepted client while an older event for the same number remains in the batch. Dead descriptors are instead placed in a reap list and closed after the batch has been processed.

- **Sessions end; accepted orders do not:**

Each connection receives a monotonically increasing session identifier (`sid`), which is also stored with its orders. On disconnection, the session is removed from `by_sid`, but its unmatched orders stay in the order book as required by §2.6. A later match still produces the trade and its public `TRADE` update; only the departed trader’s private notification is dropped.

- **Strict byte-level validation:**

Protocol messages are parsed as bytes. Numeric tokens must first match `^[0-9]+$` and must then fall within the specified range. This rejects values such as `+100`, `1_000`, decimal numbers, and Unicode digits that a looser conversion could accept.

- **Immediate small messages:**

Accepted sockets use `TCP_NODELAY`, preventing Nagle’s algorithm from delaying short market-data and execution notifications.

- **Graceful QUIT:**

After sending `QUIT`, a client calls `s.shutdown(socket.SHUT_WR)`. This sends a `FIN` for the outgoing direction while the client continues receiving. The server likewise waits for any pending bytes in `wbuf` to reach the kernel before closing the connection.

2. Experiment 1 — Listening and Connected Sockets

2.1 Technical Analysis

A listening TCP socket functions as an incoming connection acceptor: it binds to a local endpoint (`127.0.0.1:5000`), specifies a foreign wildcard (`*:*`), remains in the `LISTEN` state, and never handles application data payloads.

Each accepted client connection is allocated an independent, dedicated file descriptor with a fully qualified 4-tuple (`local_ip:5000`, `foreign_ip:client_port`) residing in the `ESTABLISHED` state. Both the listening socket and all active connections share the identical local port number; incoming packets are demultiplexed by the kernel TCP stack based on the complete 4-tuple.

2.2 Investigation Methodology & Tools

The experiment was launched using the harness, and socket allocations were inspected via FreeBSD system utilities:

```
python3 experiment.py 1
sockstat -4 | grep 5000
netstat -an -p tcp | grep 5000
```

2.3 Empirical Evidence

```
root@freebsd:~/col334_a2 # sockstat -4 | grep 5000
root python3.12 11421 3 tcp4 127.0.0.1:5000      *:*
root python3.12 11421 5 tcp4 127.0.0.1:5000      127.0.0.1:27445
root python3.12 11420 3 tcp4 127.0.0.1:27445    127.0.0.1:5000
root@freebsd:~/col334_a2 # netstat -an -p tcp | grep 5000
tcp4          0      0 127.0.0.1.5000      127.0.0.1.27445    ESTABLISHED
tcp4          0      0 127.0.0.1.27445     127.0.0.1.5000     ESTABLISHED
tcp4          0      0 127.0.0.1.5000      *:*                 LISTEN
```

Figure 1: `sockstat` and `netstat` outputs during Experiment 1, displaying the listening descriptor alongside the established client connection.

Kernel Socket Table Inspection:

```
$ sockstat -4 | grep 5000
root python3.12 4898 3 tcp4 127.0.0.1:5000      *:*
```

```
root python3.12 4898  5 tcp4  127.0.0.1:5000      127.0.0.1:28907
root python3.12 4897  3 tcp4  127.0.0.1:28907    127.0.0.1:5000
```

```
$ netstat -an -p tcp | grep 5000
tcp4  0  0  127.0.0.1.5000    127.0.0.1.28907  ESTABLISHED
tcp4  0  0  127.0.0.1.28907   127.0.0.1.5000   ESTABLISHED
tcp4  0  0  127.0.0.1.5000    *.*              LISTEN
```

Table 1: Exchange Server Socket Descriptors

File Descriptor	Socket Role	Local Address	Foreign Address	TCP State
fd=3	Listening Acceptor	127.0.0.1:5000	*:*	LISTEN
fd=5 (sid=2)	Active Connection	127.0.0.1:5000	127.0.0.1:28907	ESTABLISHED

3. Experiment 2 — Observing TCP Connection States

3.1 Technical Analysis

TCP connection lifecycles follow a strict state progression:

1. **Connection Setup:** The three-way handshake transitions endpoints from `LISTEN` / `SYN_SENT` → `SYN_RECEIVED` → `ESTABLISHED`.
2. **Orderly Teardown:** The client acts as the active closer by invoking `close()`, transmitting a `FIN` control segment. The client enters `FIN_WAIT_1`, while the server transitions to `CLOSE_WAIT` upon acknowledging the `FIN`.
3. **Server Completion:** The server reads EOF (`recv()` returning `b''`), closes its local descriptor, and sends its own `FIN`, transitioning to `LAST_ACK`.
4. **Final Closure:** Upon receiving the server's `FIN`, the client sends a final `ACK` and enters `TIME_WAIT` (retained for $2 \times \text{MSL}$ to absorb delayed duplicate segments), while the server transitions directly to `CLOSED`.

Because our event loop promptly detects EOF on read readiness, the server's duration in `CLOSE_WAIT` is minimal (measured between $504\mu\text{s}$ and $984\mu\text{s}$), completely preventing file descriptor exhaustion.

3.2 Investigation Methodology & Tools

Full packet capture on `lo0` combined with high-frequency `netstat` sampling:

```
tcpdump -i lo0 -n -s0 -w exp2.pcap 'tcp port 5000' &
python3 experiment.py 2
netstat -an -p tcp | grep 5000
tcpdump -r exp2.pcap -n -ttt -S
```

3.3 Empirical Evidence

[illegible]

Figure 2a: Repeated `netstat` sampling across connection and disconnection phases.

```

root@frebsd:~# col1334.a2 # kill K1
root@frebsd:~# col1334.a2 # 13 packets captured
13 packets received by filter
0 packets dropped by kernel
tcpdump -r exp2.pcap -ntt -s
reading from file exp2.pcap, link-type NULL (BSD loopback), snapshot length 262144
0.00:00:00.000000 IP 127.0.0.1:5000 → 127.0.0.1:5000: Flags [S], seq 16264569, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 845762809 ecr 0], length 0
0.00:00:00.000026 IP 127.0.0.1:5000 → 127.0.0.1:329498: Flags [R], seq 0, ack 642144570, win 0, length 0
0.00:00:00.100912 IP 127.0.0.1:21444 → 127.0.0.1:5000: Flags [S], seq 1671568193, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2949774629 ecr 0], length 0
0.00:00:00.000048 IP 127.0.0.1:5000 → 127.0.0.1:21441: Flags [S], seq 1565152674, ack 1671568194, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1188108887 ecr 2949774629], length 0
0.00:00:00.000019 IP 127.0.0.1:21441 → 127.0.0.1:5000: Flags [L], ack 1565152675, win 320, options [nop,nop,TS val 2949774629 ecr 1188108887], length 0
0.00:00:00.000064 IP 127.0.0.1:21441 → 127.0.0.1:5000: Flags [R], seq 1671568194, ack 1565152675, win 0, options [nop,nop,TS val 2949774629 ecr 1188108887], length 0
0.00:00:00.000030 IP 127.0.0.1:5000 → 127.0.0.1:5000: Flags [S], seq 2058073201, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3862064574 ecr 0], length 0
0.00:00:00.000010 IP 127.0.0.1:5000 → 127.0.0.1:62724: Flags [S], seq 84127483, ack 2058073200, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1029497203 ecr 3862064566], length 0
0.00:00:00.000007 IP 127.0.0.1:62724 → 127.0.0.1:5000: Flags [L], ack 84127483, win 320, options [nop,nop,TS val 3862064566 ecr 1029497203], length 0
0.00:00:10.003715 IP 127.0.0.1:62724 → 127.0.0.1:5000: Flags [F], seq 2058073200, ack 84127483, win 320, options [nop,nop,TS val 3862064574 ecr 1029497203], length 0
0.00:00:00.000030 IP 127.0.0.1:5000 → 127.0.0.1:62724: Flags [S], ack 2058073201, win 320, options [nop,nop,TS val 1029507211 ecr 3862064574], length 0
0.00:00:00.000084 IP 127.0.0.1:5000 → 127.0.0.1:62724: Flags [F], seq 84127483, ack 2058073201, win 320, options [nop,nop,TS val 1029507211 ecr 3862064574], length 0
0.00:00:00.000014 IP 127.0.0.1:62724 → 127.0.0.1:5000: Flags [L], ack 84127483, win 320, options [nop,nop,TS val 3862064574 ecr 1029507211], length 0
1) Done
root@frebsd:~# col1334.a2 #
tcpdump -i loob -n -s -w exp2.pcap tcp port 5000

```

Figure 2b: Wire-level packet capture confirming the complete four-way handshake and teardown.

Packet Trace Chronology (exp2.pcap):

```
+0.000098s  SYN 62724->5000 -> SYN/ACK -> ACK          (Handshake established)
+10.003715s [62724->5000] FIN,ACK                      (Client initiates close)
+0.000030s  [5000->62724] ACK                          (Server ACKs; enters CLOSE_WAIT)
+0.000504s  [5000->62724] FIN,ACK                      (Server closes; enters LAST ACK)
```

+0.000014s [62724->5000] ACK

(Client ACKs; enters TIME_WAIT)

Table 2: State Machine Transition Timeline

Elapsed	Wire Event	Client State	Server State
t_0	Handshake complete (SYN → SYN/ACK → ACK)	ESTABLISHED	ESTABLISHED
$t_0 + 10.0\text{ s}$	Client issues <code>close()</code> , sends FIN	FIN_WAIT_1 → FIN_WAIT_2	CLOSE_WAIT
$+30\text{ }\mu\text{s}$	Server acknowledges client FIN	FIN_WAIT_2	CLOSE_WAIT
$+504\text{ }\mu\text{s}$	Server issues <code>close()</code> , sends FIN	TIME_WAIT	LAST_ACK
$+14\text{ }\mu\text{s}$	Client acknowledges server FIN	TIME_WAIT	CLOSED

4. Experiment 3 — TCP as a Byte Stream

4.1 Technical Analysis

TCP provides a structured byte stream service without preserving application-level message boundaries. Network boundaries are determined by sender buffering, MTU limits, TCP segment sizing, and transmission timing. Application protocols running over TCP must implement explicit framing.

In this experiment, the client transmits `LOGIN experiment_trader\n` across four separate `send()` operations spaced by 200 ms. The server performs four discrete `recv()` invocations. The application framer buffers incoming bytes until the `\n` delimiter is encountered, emitting the parsed command only upon receiving the fourth segment.

4.2 Investigation Methodology & Tools

```
tcpdump -i lo0 -n -s0 -w exp3.pcap 'tcp port 5000' &
python3 experiment.py 3
tcpdump -r exp3.pcap -n -ttt -S
```

4.3 Empirical Evidence

```

root@freebsd:~/col1334_a2 # tcpdump -i lo0 -n -s0 -w exp3.pcap 'tcp port 5000' &
root@freebsd:~/col1334_a2 # tcpdump: listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
python3 experiment.py 3
=== Experiment 3: TCP as a Byte Stream ===

Started Exchange Server (PID 11319).
  0.113ms listen          fd=3 host='127.0.0.1' port=5000 backlog=4096
Exchange Server is listening.
  80.007ms accept         sid=1 fd=5 peer='127.0.0.1:19220' nconn=1 sndbuf=49032 rcvbuf=81720

Experiment client: connected from 127.0.0.1:65502
  80.088ms accept         sid=2 fd=6 peer='127.0.0.1:65502' nconn=2 sndbuf=49032 rcvbuf=81720

Sending one application-level message using several separate TCP writes...

Sent 6 bytes.
  80.140ms eof_rst        sid=1 fd=5 errno=54 ev_fflags=54
  80.173ms close          sid=1 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
  80.191ms destroy        sid=1 fd=5 nconn=1
  80.220ms recv           sid=2 fd=6 n=6 rbuf_before=0 rbuf_after=6 lines_out=0 hex='4c4f47494e20'
Sent 10 bytes.
  281.372ms recv          sid=2 fd=6 n=10 rbuf_before=6 rbuf_after=16 lines_out=0 hex='6578706572696d656e74'
Sent 7 bytes.
  483.431ms recv          sid=2 fd=6 n=7 rbuf_before=16 rbuf_after=23 lines_out=0 hex='5f747261646572'
Sent 1 bytes.
  684.034ms recv          sid=2 fd=6 n=1 rbuf_before=23 rbuf_after=0 lines_out=1 hex='0a'
  684.199ms engine        sid=2 n_msgs=1 engine_ns=25458
  684.244ms queue_out     sid=2 fd=6 n=3 pending=3 hwm=3

Investigate what the Exchange Server received.
Press Ctrl-C when you are finished.
^C
Experiment finished.
Stopping Exchange Server...
  4924.844ms eof_fin      sid=2 fd=6 ev_eof=True
  4925.175ms signal       signo=15
root@freebsd:~/col1334_a2 #

```

Figure 3a: Server execution trace detailing partial reads, buffer accumulation, and final dispatch.

```

root@freebsd:~/col1334_a2 # kill %1
root@freebsd:~/col1334_a2 # 22 packets captured
22 packets received by filter
0 packets dropped by kernel
tcpdump -r exp3.pcap -n -ttt -S
reading from file exp3.pcap, link-type NULL (BSD loopback), snapshot length 262144
00:00:00.000000 IP 127.0.0.1.26753 > 127.0.0.1.5000: Flags [S], seq 410569080, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1182648609 ecr 0], length 0
00:00:00.000028 IP 127.0.0.1.5000 > 127.0.0.1.26753: Flags [R.], seq 0, ack 410569081, win 0, length 0
00:00:00.102483 IP 127.0.0.1.19220 > 127.0.0.1.5000: Flags [S], seq 152221514, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3191004455 ecr 0], length 0
00:00:00.000043 IP 127.0.0.1.5000 > 127.0.0.1.19220: Flags [S.], seq 2669330520, ack 152221515, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2622429932 ecr 3191004455], length 0
00:00:00.000013 IP 127.0.0.1.19220 > 127.0.0.1.5000: Flags [R.], seq 152221515, ack 2669330520, win 0, options [nop,nop,TS val 3191004455 ecr 2622429932], length 0
00:00:00.000062 IP 127.0.0.1.19220 > 127.0.0.1.5000: Flags [R.], seq 152221515, ack 2669330520, win 0, options [nop,nop,TS val 3191004455 ecr 2622429932], length 0
00:00:00.000066 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [S], seq 2692795946, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3532811570 ecr 0], length 0
00:00:00.000005 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [S.], seq 1154791356, ack 2692795947, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 1455016221 ecr 3532811570], length 0
00:00:00.000006 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [R.], seq 1154791356, win 320, options [nop,nop,TS val 3532811570 ecr 1455016221], length 0
00:00:00.000062 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795947:2692795953, ack 1154791356, win 320, options [nop,nop,TS val 3532811570 ecr 1455016221], length 6
00:00:00.038466 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [R.], seq 2692795953, win 320, options [nop,nop,TS val 1455016200 ecr 3532811570], length 0
00:00:00.162590 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795953:2692795963, ack 1154791356, win 320, options [nop,nop,TS val 3532811766 ecr 1455016200], length 10
00:00:00.037511 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [R.], seq 2692795963, win 320, options [nop,nop,TS val 1455016440 ecr 3532811766], length 0
00:00:00.164443 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795963:2692795970, ack 1154791356, win 320, options [nop,nop,TS val 3532811974 ecr 1455016440], length 7
00:00:00.041018 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [R.], seq 2692795970, win 320, options [nop,nop,TS val 1455016666 ecr 3532811974], length 0
00:00:00.159561 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [P.], seq 2692795970:2692795971, ack 1154791356, win 320, options [nop,nop,TS val 3532812174 ecr 1455016666], length 1
00:00:00.000592 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [P.], seq 1154791356:1154791359, ack 2692795971, win 320, options [nop,nop,TS val 1455016825 ecr 3532812174], length 3
00:00:00.039231 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [R.], seq 1154791359, win 320, options [nop,nop,TS val 3532812214 ecr 1455016825], length 0
00:00:04.200043 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [F.], seq 2692795971, ack 1154791359, win 320, options [nop,nop,TS val 3532816414 ecr 1455016825], length 0
00:00:00.000322 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [R.], seq 2692795972, win 320, options [nop,nop,TS val 1455021065 ecr 3532816414], length 0
00:00:00.000626 IP 127.0.0.1.5000 > 127.0.0.1.65502: Flags [F.], seq 1154791359, ack 2692795972, win 320, options [nop,nop,TS val 1455021065 ecr 3532816414], length 0
00:00:00.000136 IP 127.0.0.1.65502 > 127.0.0.1.5000: Flags [R.], seq 1154791360, win 320, options [nop,nop,TS val 3532816414 ecr 1455021065], length 0
[1] Done          tcpdump -i lo0 -n -s0 -w exp3.pcap tcp port 5000
root@freebsd:~/col1334_a2 #

```

Figure 3b: Packet readback confirming four distinct PSH segments arriving over loopback.

Server Trace Analysis:

Time	Event	sid	fd	n	rbuf_pre	rbuf_post	lines	Payload	
80.220ms	recv	2	6	6	0	6	0	"LOGIN "	
281.372ms	recv	2	6	10	6	16	0	"experiment"	
483.431ms	recv	2	6	7	16	23	0	"_trader"	
684.034ms	recv	2	6	1	23	0	1	"\n"	
684.199ms	engine dispatch:	sid=2, messages=1, latency=25.4 us							

Table 3: Stream Reassembly Ledger

Chunk	Interval	Length	Cumulative Buffer	Parser Status
1	80 ms	6 B	LOGIN (6 B)	Incomplete (0 emitted)
2	281 ms	10 B	LOGIN experiment (16 B)	Incomplete (0 emitted)

Chunk	Interval	Length	Cumulative Buffer	Parser Status
3	483 ms	7 B	LOGIN experiment_trader (23 B)	Incomplete (0 emitted)
4	684 ms	1 B	Frame complete on \n	Emitted 1 command (OK returned)

Offline unit tests in `tests/test_all.py` further verify stream reassembly invariance across all 47 possible 2-way split permutations and 1,000 randomized multi-fragment cuts.

5. Experiment 4 — One Client Should Not Stall the Others

5.1 Technical Analysis

In an unmultiplexed, blocking architecture, a client that connects and halts mid-message causes the server process or thread to block inside `recv()`. The server becomes incapable of accepting new connections or servicing other clients.

Using an event-driven kqueue architecture, the server waits exclusively inside `kevent()`. A descriptor is only returned when the operating system indicates read readiness. While Client 1 sits idle without terminating its line, the server services Client 2 instantaneously (elapsed time: 0.003 s).

5.2 Comparative Investigation (A/B Control)

To empirically verify the blocking mechanism, a baseline blocking server (`src/naive_server.py`) was evaluated under identical conditions:

```
python3 experiment.py 4          # Production kqueue server
EXCH_NAIVE=1 python3 experiment.py 4  # Blocking baseline control
ps -o pid,tid,wchan,state -p <pid>
```

5.3 Empirical Evidence

```
root@freebsd:~/col334_a2 # pgrep -f server.py
11492
root@freebsd:~/col334_a2 #
root@freebsd:~/col334_a2 # ps -o pid,tid,wchan,state -H -p 11492
  PID   LWP  WCHAN  STAT
11492 100187 kqread  Is
root@freebsd:~/col334_a2 #
```

Figure 4a: Production server execution: Client 2 completes in 0.003 s with kernel wait channel `wchan=kqread`.

```

root@freebsd:~/col334_a2 # pgrep -f server.py
11499
root@freebsd:~/col334_a2 # pgrep -f server.py 11499
11499
root@freebsd:~/col334_a2 # ps -o pid,tid,wchan,state -H -p 11499
  PID    LWP  WCHAN  STAT
11499 100187 sbwait  Is
root@freebsd:~/col334_a2 # █

```

Figure 4b: Blocking baseline execution: Client 2 times out after 5.07 s with kernel wait channel `wchan=sbwait`.

Table 4: Concurrency Performance & Wait Channel Comparison

Metric / Parameter	Production Server (<code>server.py</code>)	Blocking Baseline (<code>naive_server.py</code>)
I/O Mechanism	Non-blocking <code>select.kqueue()</code>	Blocking POSIX sockets
Kernel Wait Channel (<code>wchan</code>)	<code>kqread</code> (waiting in <code>kevent</code>)	<code>sbwait</code> (blocked on socket buffer read)
Client 2 Response	OK	None (Timed out)
Client 2 Elapsed Time	0.003 s	5.072 s
Client 1 Socket Queue (<code>Recv-Q</code>)	0 (Buffered in userspace <code>rbuf</code>)	0 (Held blocked in kernel <code>recv</code>)

The wait channel confirms the root behavior: `sbwait` reflects thread suspension on an individual socket buffer, whereas `kqread` indicates scalable multiplexing over all registered descriptors.

6. Experiment 5 — Multiple Clients and I/O Multiplexing

6.1 Technical Analysis

I/O multiplexing decouples connection presence from CPU servicing cost. Across five concurrently established connections where only three submit data (Clients 1, 3, 5), the event loop is dispatched exclusively for descriptors possessing unread socket buffers. Idle connections consume zero CPU time, exhibiting $O(\text{ready})$ rather than $O(\text{registered})$ scaling.

6.2 Investigation Methodology & Tools

```

python3 experiment.py 5
netstat -an -p tcp | grep '\.5000 '

```

6.3 Empirical Evidence

```
root@freebsd:~/col334_a2 # python3 experiment.py 5
=== Experiment 5: Multiple Clients and I/O Multiplexing (Optional) ===

Started Exchange Server (PID 11341).
  0.118ms listen      fd=3 host='127.0.0.1' port=5000 backlog=4096
Exchange Server is listening.
  79.138ms accept      sid=1 fd=5 peer='127.0.0.1:28635' nconn=1 sndbuf=49032 rcvbuf=81720
  79.173ms eof_rst      sid=1 fd=5 errno=54 ev_fflags=54
  79.188ms close      sid=1 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
  79.204ms destroy      sid=1 fd=5 nconn=0

Creating five simultaneous TCP connections...

Client 1: connected from 127.0.0.1:11117
  79.396ms accept      sid=2 fd=5 peer='127.0.0.1:11117' nconn=1 sndbuf=49032 rcvbuf=81720
Client 2: connected from 127.0.0.1:48451
  79.554ms accept      sid=3 fd=6 peer='127.0.0.1:48451' nconn=2 sndbuf=49032 rcvbuf=81720
Client 3: connected from 127.0.0.1:61204
  79.651ms accept      sid=4 fd=7 peer='127.0.0.1:61204' nconn=3 sndbuf=49032 rcvbuf=81720
Client 4: connected from 127.0.0.1:20623
Client 5: connected from 127.0.0.1:46858
  79.688ms accept      sid=5 fd=8 peer='127.0.0.1:20623' nconn=4 sndbuf=49032 rcvbuf=81720
  79.706ms accept      sid=6 fd=9 peer='127.0.0.1:46858' nconn=5 sndbuf=49032 rcvbuf=81720
Clients 1, 3, and 5 will send application messages.
Clients 2 and 4 will remain idle.

Client 1: sent application data.
  79.798ms recv      sid=2 fd=5 n=15 rbuf_before=0 rbuf_after=0 lines_out=1 hex='4c4f47494e20636c69656e745f310a'
  79.890ms engine      sid=2 n_msgs=1 engine_ns=5375
  79.899ms queue_out      sid=2 fd=5 n=3 pending=3 hwm=3
Client 3: sent application data.
  581.748ms recv      sid=4 fd=7 n=15 rbuf_before=0 rbuf_after=0 lines_out=1 hex='4c4f47494e20636c69656e745f330a'
  582.189ms engine      sid=4 n_msgs=1 engine_ns=27458
  582.255ms queue_out      sid=4 fd=7 n=3 pending=3 hwm=3
Client 5: sent application data.
  1083.395ms recv      sid=6 fd=9 n=15 rbuf_before=0 rbuf_after=0 lines_out=1 hex='4c4f47494e20636c69656e745f350a'
  1083.641ms engine      sid=6 n_msgs=1 engine_ns=27333
  1083.682ms queue_out      sid=6 fd=9 n=3 pending=3 hwm=3

The experiment is now in the observation phase.
Investigate which connections were active.
Press Ctrl-C when you are finished.
Stopping Exchange Server...
  16586.312ms signal      signo=15
root@freebsd:~/col334_a2 #
```

Figure 5a: Event loop execution trace confirming read events trigger solely for active clients.

[illegible]

Figure 5b: System connection table showing all five connections concurrently established.

Table 5: Connection Readiness vs. Service Activity

Session ID	File Descriptor	Foreign Port	Client Role	recv() Activity	Recv-Q Status
sid=2	fd=5	11117	Client 1	Active (1 read, OK sent)	3 B (Client unread reply)
sid=3	fd=6	48451	Client 2	None (Idle)	0 B / 0 B
sid=4	fd=7	61204	Client 3	Active (1 read, OK sent)	3 B (Client unread reply)
sid=5	fd=8	20623	Client 4	None (Idle)	0 B / 0 B
sid=6	fd=9	46858	Client 5	Active (1 read, OK sent)	3 B (Client unread reply)

7. Experiment 6 — FIN vs. RST: Orderly and Abrupt Termination

7.1 Technical Analysis

- **Orderly Termination (FIN):** A synchronized protocol event initiated via `shutdown(SHUT_WR)` or `close()`. The FIN control segment carries an incremented sequence number, guarantees in-order delivery of preceding data, allows graceful half-close, and places the active closer in `TIME_WAIT`. The peer application detects closure via EOF (`recv()` returns `b''`).
- **Abrupt Termination (RST):** An immediate connection abort initiated via `SO_LINGER{on=1, timeout=0}` or process termination errors. The RST segment is unsequenced, discards queued network buffers in both directions, generates no `TIME_WAIT` state, and immediately surfaces as `ECONNRESET` (errno 54).

Under FreeBSD `kqueue`, socket errors are delivered directly within the event structure (`ev.fflags`), allowing discrimination between normal EOF and an abortive reset before issuing `recv()`.

7.2 Investigation Methodology & Tools

```
tcpdump -i lo0 -n -s0 -w exp6.pcap 'tcp port 5000' &
python3 experiment.py 6
tcpdump -r exp6.pcap -n -ttt -S
```

7.3 Empirical Evidence

```

root@freebsd:~/col334_a2 # netstat -an -p tcp | grep '\.5000'
root@freebsd:~/col334_a2 # kill %1
root@freebsd:~/col334_a2 # 34 packets captured
34 packets received by filter
0 packets dropped by kernel
tcpdump -r exp6.pcap -n -ttt -S
reading from file exp6.pcap, link-type NULL (BSD loopback), snapshot length 262144
00:00:00.000000 IP 127.0.0.1.57843 > 127.0.0.1.5000: Flags [S], seq 672047514, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3215072442 ecr 0], length 0
00:00:00.000005 IP 127.0.0.1.5000 > 127.0.0.1.57843: Flags [R.], seq 0, ack 672047515, win 0, length 0
00:00:00.104502 IP 127.0.0.1.12541 > 127.0.0.1.5000: Flags [S], seq 2836097739, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 550528494 ecr 0], length 0
00:00:00.000037 IP 127.0.0.1.5000 > 127.0.0.1.12541: Flags [S.], seq 903231691, ack 2836097740, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3047162408 ecr 550528494], length 0
00:00:00.000013 IP 127.0.0.1.12541 > 127.0.0.1.5000: Flags [F.], seq 903231692, win 320, options [nop,nop,TS val 550528494 ecr 3047162408], length 0
00:00:00.000073 IP 127.0.0.1.12541 > 127.0.0.1.5000: Flags [R.], seq 2836097740, ack 903231692, win 0, options [nop,nop,TS val 550528494 ecr 3047162408], length 0
00:00:00.000195 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [S], seq 4042106586, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2731617098 ecr 0], length 0
00:00:00.000004 IP 127.0.0.1.5000 > 127.0.0.1.22932: Flags [S.], seq 1104886719, ack 4042106587, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 742386677 ecr 2731617098], length 0
00:00:00.000005 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [F.], seq 1104886720, win 320, options [nop,nop,TS val 2731617098 ecr 742386677], length 0
00:00:00.000030 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [F.], seq 4042106587, ack 1104886720, win 320, options [nop,nop,TS val 2731617098 ecr 742386677], length 0
00:00:00.000002 IP 127.0.0.1.5000 > 127.0.0.1.22932: Flags [F.], seq 1104886720, ack 4042106588, win 320, options [nop,nop,TS val 742386677 ecr 2731617098], length 0
00:00:00.000002 IP 127.0.0.1.22932 > 127.0.0.1.5000: Flags [F.], seq 1104886721, win 320, options [nop,nop,TS val 2731617098 ecr 742386677], length 0
00:00:10.003722 IP 127.0.0.1.34505 > 127.0.0.1.5000: Flags [S], seq 3508927132, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3479980373 ecr 0], length 0
00:00:00.000217 IP 127.0.0.1.5000 > 127.0.0.1.34505: Flags [S.], seq 2987702093, ack 3508927133, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 7946555570 ecr 0], length 0
00:00:00.000039 IP 127.0.0.1.34505 > 127.0.0.1.5000: Flags [F.], seq 2987702094, win 320, options [nop,nop,TS val 3479980373 ecr 7946555570], length 0
00:00:00.000218 IP 127.0.0.1.34505 > 127.0.0.1.5000: Flags [R.], seq 3508927133, ack 2987702094, win 0, options [nop,nop,TS val 3479980373 ecr 7946555570], length 0
00:00:14.312347 IP 127.0.0.1.39505 > 127.0.0.1.5000: Flags [S], seq 2808249619, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2808249619 ecr 0], length 0
00:00:00.000026 IP 127.0.0.1.5000 > 127.0.0.1.39505: Flags [R.], seq 0, ack 2808249619, win 0, length 0
00:00:00.101961 IP 127.0.0.1.45901 > 127.0.0.1.5000: Flags [S], seq 3200946643, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 991525530 ecr 0], length 0
00:00:00.000034 IP 127.0.0.1.5000 > 127.0.0.1.45901: Flags [S.], seq 5248096697, ack 3200946644, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2000548351 ecr 991525530], length 0
00:00:00.000012 IP 127.0.0.1.45901 > 127.0.0.1.5000: Flags [F.], seq 5248096698, win 320, options [nop,nop,TS val 991525530 ecr 2000548351], length 0
00:00:00.000063 IP 127.0.0.1.45901 > 127.0.0.1.5000: Flags [R.], seq 3200946644, ack 5248096698, win 0, options [nop,nop,TS val 991525530 ecr 2000548351], length 0
00:00:00.000002 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [S], seq 2916099747, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 263287886 ecr 0], length 0
00:00:00.000005 IP 127.0.0.1.5000 > 127.0.0.1.16177: Flags [S.], seq 423294695, ack 2916099748, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3174048641 ecr 263287886], length 0
00:00:00.000006 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [F.], seq 423294696, win 320, options [nop,nop,TS val 263287886 ecr 3174048641], length 0
00:00:00.000024 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [F.], seq 2916099748, ack 423294696, win 320, options [nop,nop,TS val 263287886 ecr 3174048641], length 0
00:00:00.000002 IP 127.0.0.1.5000 > 127.0.0.1.16177: Flags [F.], seq 2916099749, win 320, options [nop,nop,TS val 3174048641 ecr 263287886], length 0
00:00:00.000003 IP 127.0.0.1.16177 > 127.0.0.1.5000: Flags [F.], seq 423294697, win 320, options [nop,nop,TS val 263287886 ecr 3174048641], length 0
00:00:10.001400 IP 127.0.0.1.50346 > 127.0.0.1.5000: Flags [S], seq 1605044384, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 212167698 ecr 0], length 0
00:00:00.000037 IP 127.0.0.1.5000 > 127.0.0.1.50346: Flags [S.], seq 1605044385, win 320, options [nop,nop,TS val 212167698 ecr 1433121945], length 0
00:00:00.000200 IP 127.0.0.1.50346 > 127.0.0.1.5000: Flags [R.], seq 1605044385, win 0, options [nop,nop,TS val 212167698 ecr 1433121945], length 0
[1] Done tcpdump -i lo0 -n -s0 -w exp6.pcap tcp port 5000
root@freebsd:~/col334_a2 #

```

Figure 6a: Packet capture distinguishing four-way FIN handshake (Part A) from single RST segment (Part B).

```

root@freebsd:~/col334_a2 # tcpdump -i lo0 -n -s0 -w exp6.pcap 'tcp port 5000' &
root@freebsd:~/col334_a2 # tcpdump: listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
python3 experiment.py 6
=== Experiment 6: FIN vs. RST: Orderly and Abrupt Connection Termination ===

Started Exchange Server (PID 11374).
  0.113ms listen fd=3 host='127.0.0.1' port=5000 backlog=4096
Exchange Server is listening.
  78.060ms accept sid=1 fd=5 peer='127.0.0.1:45901' nconn=1 sndbuf=49032 rcvbuf=81720

  78.102ms eof_rst sid=1 fd=5 errno=54 ev_fflags=54
Part A: orderly connection termination (FIN).
  78.142ms close sid=1 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
Orderly client: connected from 127.0.0.1:16177
The client will shut down its sending direction.
Investigate the TCP traffic and connection state.
  78.193ms destroy sid=1 fd=5 nconn=0
  78.295ms accept sid=2 fd=5 peer='127.0.0.1:16177' nconn=1 sndbuf=49032 rcvbuf=81720
  78.320ms eof_fin sid=2 fd=5 ev_eof=True
  78.333ms close sid=2 fd=5 why='FIN' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
  78.355ms destroy sid=2 fd=5 nconn=0

Part B: abortive connection termination (RST).
Abrupt client: connected from 127.0.0.1:50346
The client will now close abortively.
Investigate the TCP traffic and connection state.

The Exchange Server will remain running for 15 seconds.
10080.108ms accept sid=3 fd=5 peer='127.0.0.1:50346' nconn=1 sndbuf=49032 rcvbuf=81720
10080.312ms eof_rst sid=3 fd=5 errno=54 ev_fflags=54
10080.565ms close sid=3 fd=5 why='RST' role='untyped' recvs=0 bytes_in=0 queued=0 sent=0 eagain=0 wbuf_hwm=0 orders_surviving=0
10080.746ms destroy sid=3 fd=5 nconn=0

Experiment finished.
Stopping Exchange Server...
25084.603ms signal signo=15

```

Figure 6b: Server logs detailing `eof_fin` (Part A) versus `eof_rst` with `errno 54` (Part B).

Wire Event Comparison:

Part A (Orderly FIN - Port 16177):

```

16177 > 5000: Flags [F.] seq 1 ack 1 (Client SHUT_WR)
5000 > 16177: Flags [.] ack 2 (Server ACK)
5000 > 16177: Flags [F.] seq 1 ack 2 (Server FIN)
16177 > 5000: Flags [.] ack 2 (Client ACK)

```

Part B (Abrupt RST - Port 50346):

```

50346 > 5000: Flags [R.] seq 1 win 0 (Immediate abortive reset)

```

Table 6: Teardown Mechanism Characteristics

Dimension	Orderly Termination (FIN)	Abrupt Termination (RST)
Packet Sequence	4-way exchange ([F.], [F.], [F.], [F.])	Single abortive packet ([R.])
recv() Application Signal	Returns empty bytes (b'')	Raises <code>ConnectionResetError</code>
FreeBSD Kernel Errno	None (0)	54 (<code>ECONNRESET</code>)
Kqueue Event Notification	<code>ev.flags & KQ_EV_EOF</code>	<code>ev.fflags == 54</code>
TIME_WAIT State	Enforced on active closer	Completely bypassed
Server State Effect	Closed cleanly; surviving peers unaffected	Closed cleanly; surviving peers unaffected

8. Experiment 7 — Backpressure and the Slow Receiver

8.1 Technical Analysis & Theoretical Capacity

When a subscriber ceases reading from its socket, incoming bytes accumulate in its receive buffer. Once full, the receiver advertises a zero-window (`win 0`), halting TCP transmissions. Continued application write attempts then saturate the server’s send buffer, causing non-blocking `send()` calls to fail with `EWOULDBLOCK` / `EAGAIN` (errno 35).

A properly decoupled server buffers excess messages in userspace and registers write notification filters (`KQ_FILTER_WRITE`) for the backlogged socket, ensuring other active clients and the core matching engine continue operating uninhibited.

8.2 Initial Arithmetic vs. FreeBSD Loopback Behavior

In standard `experiment.py` 7:

- Message size: `TRADE_JNST 1 238\n` = 17 bytes.
- Order volume: 5,000 trade iterations = 85,003 bytes per subscriber (including 3 B `OK\n`).
- FreeBSD loopback socket buffers: default `sendspace` = 32,768 B, `recvspace` = 65,536 B (aggregate capacity $\approx 98,304$ B).

Because $85,003 < 98,304$, the data fits entirely within kernel buffers. Run A confirms this: `send_would_block` = 0, with the slow subscriber’s kernel `Recv-Q` absorbing all 83,232 bytes while the active subscriber maintained `Recv-Q` near zero.

[illegible]

Figure 7a: *netstat* and *ps* sampling showing slow subscriber *Recv-Q* scaling up to 83,232 B while server remains in *kqread*.

Table 7: Run A Kernel Buffer Accumulation

Sample	Fast Subscriber Recv-Q	Slow Subscriber Recv-Q	Server wchan
1	17 B	629 B	kqread
2	17 B	3,757 B	kqread
3	0 B	22,083 B	kqread
4	0 B	34,816 B	kqread
5	17 B	54,740 B	kqread
6	0 B	74,834 B	kqread
7	17 B	83,232 B	kqread

8.3 Platform Investigation: FreeBSD Loopback Flow Control

To induce backpressure within the default 5,000-message quota, buffer sizes were restricted via `SO_RCVBUF=8192` and `SO_SNDBUF=1024`. However, *Recv-Q* continued to climb past limits without triggering backpressure.

An isolated standalone diagnostic (`src/tools/sb_probe.py`) revealed that on FreeBSD’s loopback interface (`lo0`), when a receiving process never reads, the TCP stack allocates kernel mbuf clusters rather than enforcing strict window clamping:

```

16777232      986896      1.43
17825792      1048576     1.52
18874369      1110257     1.62
19922946      1171938     1.71
20971523      1233619     1.80
22020100      1295300     1.88
23068677      1356981     1.97
24117254      1418662     2.06
25165831      1480343     2.15
26214408      1542024     2.23
27262985      1603705     2.32
28311562      1665386     2.41
29360139      1727067     2.50
30408716      1788748     2.59
31457293      1850429     2.68
32505870      1912110     2.78
33554447      1973791     2.87
34603024      2035472     2.96
35651584      2097152     3.05
36700161      2158833     3.13
37748738      2220514     3.21
38797315      2282195     3.30
39845892      2343876     3.39
40894469      2405557     3.48
41943046      2467238     3.57
42991623      2528919     3.65
44040200      2590600     3.75
45088777      2652281     3.84
46137354      2713962     3.93
47185931      2775643     4.02
48234508      2837324     4.11
49283085      2899005     4.21
50331662      2960686     4.30
51380239      3022367     4.38
52428816      3084048     4.46
53477376      3145728     4.54
54525953      3207409     4.63
55574530      3269090     4.72
56623107      3330771     4.81
57671684      3392452     4.89
58720261      3454133     4.97
59768838      3515814     5.05
60817415      3577495     5.13
61865992      3639176     5.21
62914569      3700857     5.30
63963146      3762538     5.39
65011723      3824219     5.47
66060300      3885900     5.56
67108877      3947581     5.65

=====
blaster RESULT
=====
PATH CAPACITY (bytes accepted before sustained stall) : 67108877
messages      : 3947581
first EWOULDBLOCK at : NEVER -- no backpressure at any point
EWOULDBLOCK count : 0
SO_SNDBUF in force : 49032

Capacity should be roughly own_sndbuf + peer_rcvbuf. If it is
orders of magnitude larger, the peer's sb_hiwat is not gating
acceptance -- which is the anomaly under investigation.

holding the connection open; Ctrl-C when the victim is done.
[]

root@freebsd:~/col334_a2 # python3 src/tools/sb_probe.py --client 127.0.0.1 6000
victim: connected. local port=11305 SO_RCVBUF=81720 (requested None)
victim: NEVER calling recv(). Ctrl-C to drain and count.

elapsed_s      FIONREAD      SO_RCVBUF
0.0            0            81720
1.0            11728402     122580
2.0            23414032     122580
3.0            35118396     122580
4.0            47004930     122580
5.0            59126731     122580
6.0            67108877     122580
7.0            67108877     122580
8.0            67108877     122580
9.0            67108877     122580
10.0           67108877     122580
11.0           67108877     122580
12.0           67108877     122580
13.0           67108877     122580
14.0           67108877     122580
15.0           67108877     122580
16.0           67108877     122580
17.0           67108877     122580
18.0           67108877     122580
19.0           67108877     122580
20.0           67108877     122580
21.1           67108877     122580
22.1           67108877     122580
23.1           67108877     122580
24.1           67108877     122580
25.1           67108877     122580
26.1           67108877     122580
27.1           67108877     122580
28.1           67108877     122580
29.1           67108877     122580
30.1           67108877     122580
31.1           67108877     122580
32.1           67108877     122580
33.1           67108877     122580
34.1           67108877     122580
35.1           67108877     122580
36.1           67108877     122580
37.1           67108877     122580
38.1           67108877     122580
39.1           67108877     122580
40.1           67108877     122580
41.1           67108877     122580
42.1           67108877     122580
43.1           67108877     122580
44.1           67108877     122580
45.1           67108877     122580
46.1           67108877     122580
47.1           67108877     122580
48.1           67108877     122580
49.1           67108877     122580
50.2           67108877     122580
51.2           67108877     122580
52.2           67108877     122580
53.2           67108877     122580
54.2           67108877     122580
55.2           67108877     122580
56.2           67108877     122580
57.2           67108877     122580
58.2           67108877     122580
59.2           67108877     122580
60.2           67108877     122580

^C
victim: draining. FIONREAD said 67108877 readable.

=====
victim RESULT
=====
FIONREAD before drain : 67108877 B
ACTUALLY DRAINED      : 67108877 B
drain took             : 1.13 s
SO_RCVBUF start / end : 81720 / 122580
root@freebsd:~/col334_a2 #

```

Figure 7b: `sb_probe.py` diagnostic on FreeBSD: 67 MB absorbed unread without stall (in contrast to Linux, which stalls at 17 KB).

```

root@freebsd:~/col334_a2 # tcpdump -r exp7e.pcap -n 'src port 12398' | head -3
reading from file exp7e.pcap, link-type NULL (BSD loopback), snapshot length 96
18:24:54.285291 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [S], seq 3741972962, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2978424221 ecr 0], length 0
18:24:54.285430 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [P], seq 226082803, win 320, options [nop,nop,TS val 2978424221 ecr 689218545], length 0
18:24:54.285476 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [P], seq 0:15, ack 1, win 320, options [nop,nop,TS val 2978424221 ecr 689218545], length 15
tcpdump: Unable to write output: Broken pipe
root@freebsd:~/col334_a2 # tcpdump -r exp7e.pcap -n 'src port 12398' | tail -3
reading from file exp7e.pcap, link-type NULL (BSD loopback), snapshot length 96
18:28:13.463690 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [F], seq 11900004, win 320, options [nop,nop,TS val 2978623401 ecr 689417685], length 0
18:28:30.160716 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [F], seq 15, ack 11900004, win 320, options [nop,nop,TS val 2978640181 ecr 689417685], length 0
18:28:30.193722 IP 127.0.0.1.12398 > 127.0.0.1.5000: Flags [F], seq 11900005, win 320, options [nop,nop,TS val 2978640131 ecr 689434455], length 0
root@freebsd:~/col334_a2 #

```

Figure 7c: Wire inspection on `exp7e.pcap` showing subscriber maintaining a constant advertised window (`win 320 = 81,920 B`) across 11.9 MB.

8.4 High-Volume Verification: Run E

To evaluate genuine backpressure handling under production-scale loads, `exp7_load.py` was executed with 700,000 matched order pairs (stock sysctl settings):

```

root@freebsd:~/col334_a2 # ls -la ~/col334_a2/exp7e_trace.jsonl ~/col334_a2/exp7e.pcap
-rw-r--r--  1 root wheel 445086643 Sep  6 18:28 /root/col334_a2/exp7e.pcap
-rw-r--r--  1 root wheel 443652856 Sep  6 18:28 /root/col334_a2/exp7e_trace.jsonl
root@freebsd:~/col334_a2 # python3 src/tools/exp7_report.py exp7e_trace.jsonl
=====
Experiment 7 trace report: exp7e_trace.jsonl
=====

-- event counts -----
queue_out          3500003
engine             1400003
recv               360
send_would_block   66
accept             3
eof_fin            3
close              3
destroy            3
listen             1
write_enable       1
write_disable      1

-- connections (kernel buffers actually in force) -----
sid  fd  peer                SO_SNDBUF  SO_RCVBUF
1    6    127.0.0.1:12398        2048       81720
2    7    127.0.0.1:35156        2048       81720
3    8    127.0.0.1:54139        2048       81720

-- P1 backpressure onset -----
first send_would_block : t=93102.525 ms  sid=3
send_would_block/sid   : {3: 66}
partial_write/sid      : {}
engine calls before onset: 534524

-- P2 engine decoupling (engine_ns, microseconds) -----
window      n      p50      p90      p99      p99.9      max
pre-onset   534524  4.3      8.0      62.6     148.0     47054.2
post-onset  865479   3.9      6.6      55.0     121.0     64416.1

ratio post/pre      : p50 x0.91  p99 x0.88
ASSERTION: both ratios must be O(1). A ratio >> 1 would mean the
            matching engine slowed down once a consumer stopped
            reading, i.e. head-of-line blocking.

throughput (engine calls / s)
pre-onset      6550/s over 81.61 s
post-onset     7364/s over 117.52 s

-- P3 per-connection isolation -----
sid  queued_B  TRADEs(17B)  wbuf_hwm_B  max_pend
1    11900003   700000       17          17
2    28151893   10          23          23
3    26737003   0           1474         1474

ASSERTION: the non-reading subscriber shows wbuf_hwm >> 17 while
            every other sid stays at or near 17 (one message).
            That is per-connection isolation: the backlog is
            charged to the slow socket, not to the server.

-- close events -----
sid=1  why=FIN      queued=11900003  sent=11900003  eagain=0  hwm=17
sid=2  why=FIN      queued=28151893  sent=28151893  eagain=0  hwm=23
sid=3  why=FIN      queued=26737003  sent=26737003  eagain=66  hwm=1474

root@freebsd:~/col334_a2 #

```

Figure 7d: *exp7_report.py* output displaying backpressure onset, per-session statistics, and engine latency.

Table 8: High-Volume Session Summary (Run E)

Session	Role	Total Queued	Peak Userspace Backlog (wbuf_hwm)	EAGAIN Events
sid=1	Subscriber (Never reads)	11,900,003 B	17 B	0
sid=2	Buyer (Trader)	28,151,893 B	23 B	0
sid=3	Seller (Trader)	26,737,003 B	1,474 B	66

The 66 EAGAIN events occurred on `sid=3` due to batch-write bursts. The server correctly responded by arming `KQ_FILTER_WRITE`, buffering the excess in userspace, and resuming writes without blocking.

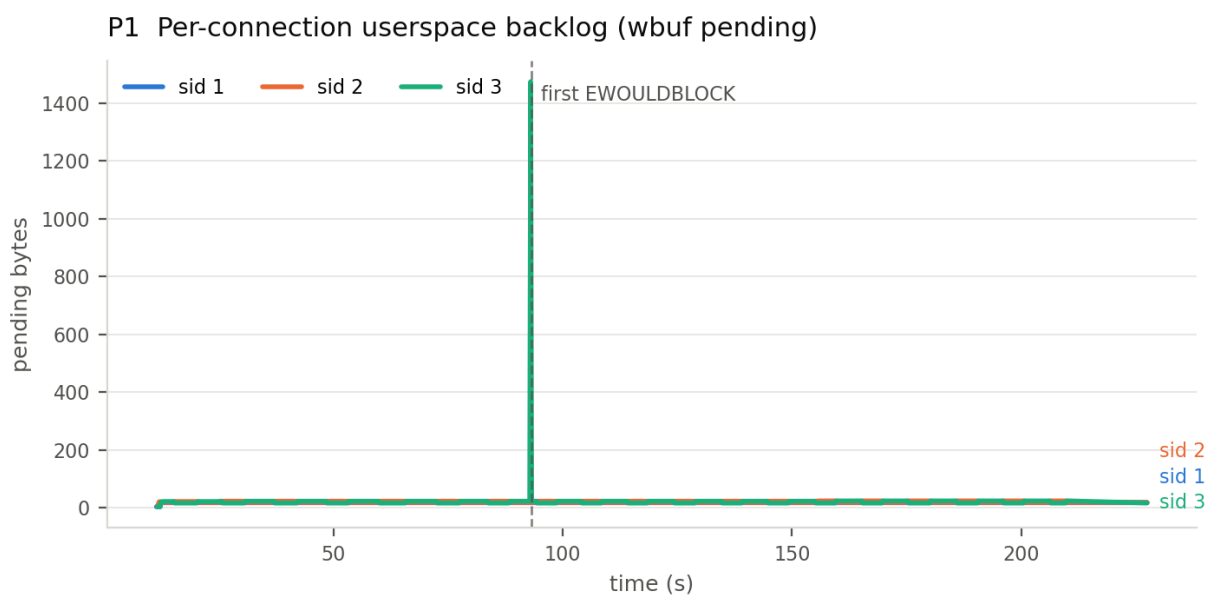


Figure 7e: Per-connection userspace backlog over time, showing transient buffering on `sid=3` with zero impact on other sessions.

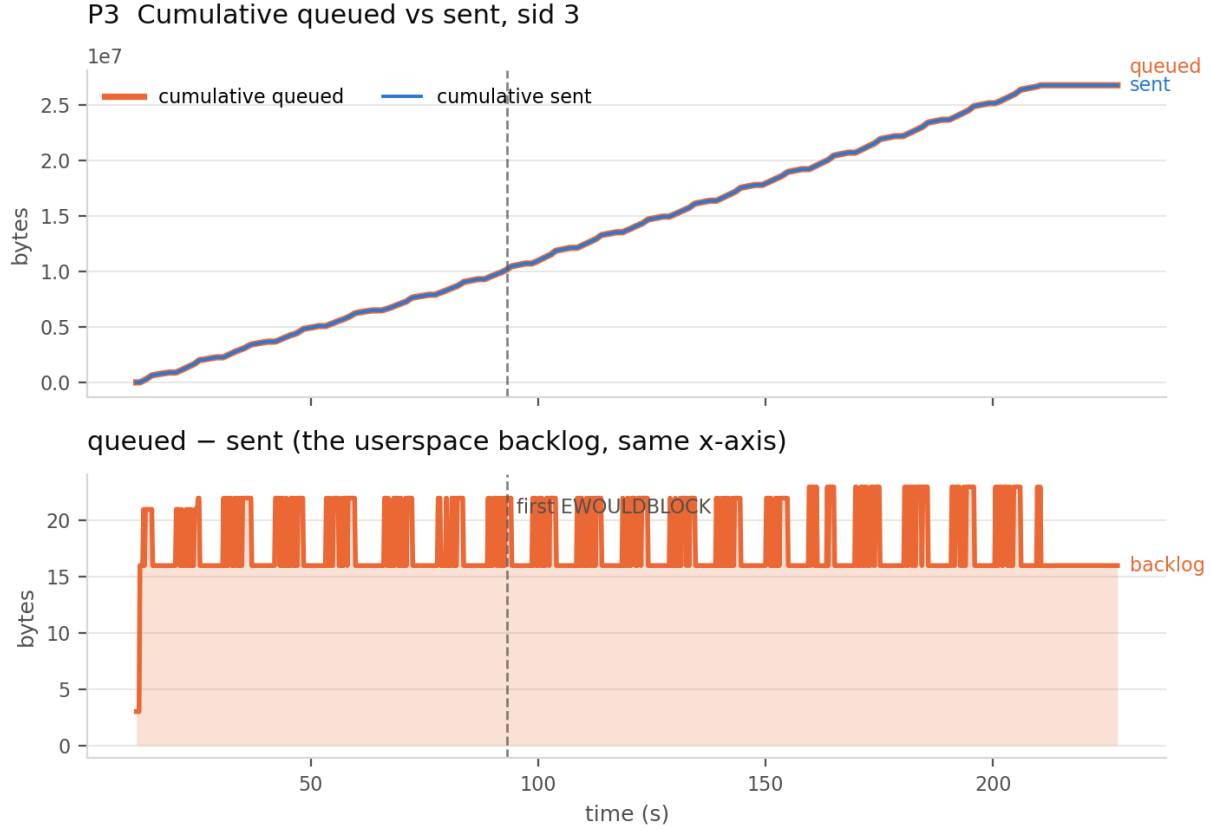


Figure 7f: Cumulative transmission volume vs. transient backlog delta for $sid=3$.

8.5 Matching Engine Decoupling Proof

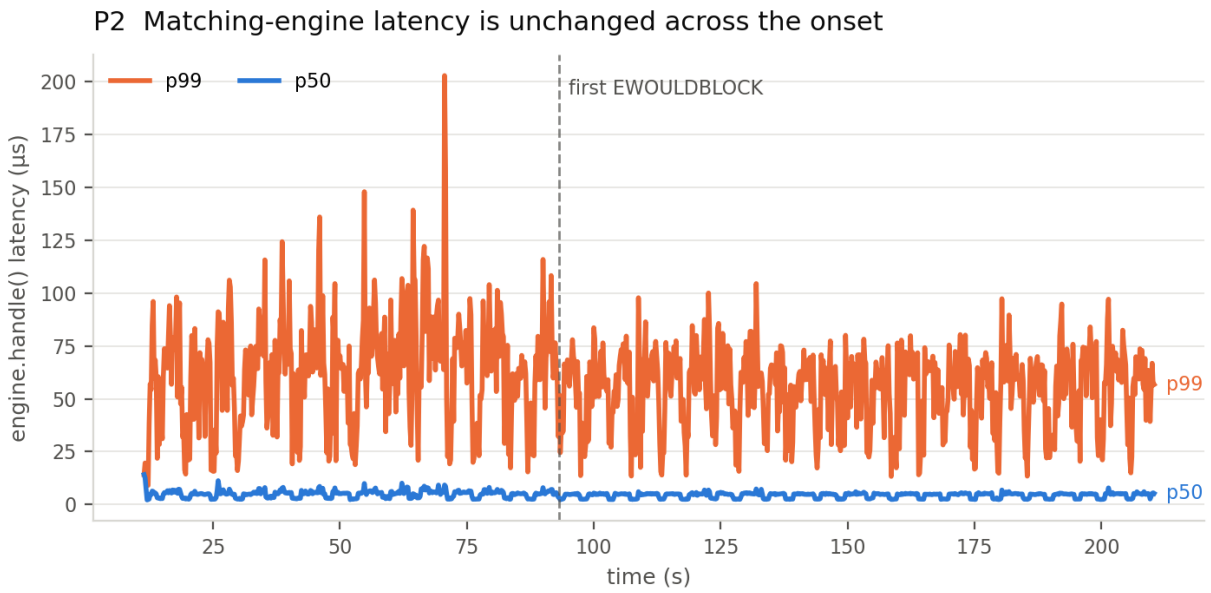


Figure 7g: Matching engine execution latency distribution across pre-onset and post-onset phases.

Table 9: Matching Engine Latency Distribution Across Backpressure Onset

Window	Order Operations	p50 Latency	p90 Latency	p99 Latency	Max Latency	Throughput
Pre-Onset	534,524	4.3 μ s	8.0 μ s	62.6 μ s	47.0 ms	6,550 ops/s
Post-Onset	865,479	3.9 μ s	6.6 μ s	55.0 μ s	64.4 ms	7,364 ops/s

Engine latency remained constant across the backpressure boundary (p50 ratio: 0.91, p99 ratio: 0.88), providing empirical proof that network write stalls do not impede order processing.

9. Experiment 8 — Unexpected Client Disconnection

9.1 Technical Analysis

When a client process is terminated abruptly via `SIGKILL`, the process is unable to execute user-space shutdown logic. However, the operating system kernel reclaims all open file descriptors on process death, transmitting an orderly `FIN` segment.

The Exchange Server detects client loss reactively upon the next event loop iteration. Disconnection of one client must not interrupt active trading between surviving participants. Furthermore, resting orders submitted prior to termination must persist in the book and remain eligible for execution (§2.6).

9.2 Investigation Methodology & Tools

```
tcpdump -i lo0 -n -s0 -w exp8.pcap 'tcp port 5000' &
python3 experiment.py 8
tcpdump -r exp8.pcap -n -ttt -S | grep <killed_port>
```

9.3 Empirical Evidence

```
The Market-Data Client process will now disappear unexpectedly.
3933.263ms eof_fin      sid=5 fd=8 ev_eof=True
3933.498ms close        sid=5 fd=8 why='FIN' role='market-data' recvs=1 bytes_in=15 queued=343 sent=343 eagain=0 wbuf_hwm=17 orders_surviving=0
3934.054ms destroy      sid=5 fd=8 nconn=3
The client process has disappeared.
Generating additional traffic.
```

Figure 8a: Server logs verifying client termination detection, session cleanup, and continued trade execution.

```

root@freedsd:~/col334_a2 # tcpdump: listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
kill %1
784 packets captured
784 packets received by filter
0 packets dropped by kernel
root@freedsd:~/col334_a2 # tcpdump -r exp8.pcap -n -ttt -S | grep 15960
reading from file exp8.pcap, link-type NULL (BSD loopback), snapshot length 262144
00:00:00.000784 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [S], seq 1521756197, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2869737745 ecr 0], length 0
00:00:00.000804 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [S.], seq 2819235974, ack 1521756198, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2148292229 ecr 2869737745], length 0
00:00:00.000813 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819235975, win 320, options [nop,nop,TS val 2869737745 ecr 2148292229], length 0
00:00:00.000820 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 1521756198:1521756213, ack 2819235975, win 320, options [nop,nop,TS val 2869737745 ecr 2148292229], length 15
00:00:00.000834 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819235975:2819235978, ack 1521756213, win 320, options [nop,nop,TS val 2148292229 ecr 2869737745], length 3
00:00:00.000785 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819235978, win 320, options [nop,nop,TS val 2869737786 ecr 2148292229], length 0
00:00:00.000820 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819235978:2819235995, ack 1521756213, win 320, options [nop,nop,TS val 2148293281 ecr 2869737786], length 17
00:00:00.0008571 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819235995, win 320, options [nop,nop,TS val 2869738757 ecr 2148293281], length 0
00:00:00.000834 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819235995:2819236012, ack 1521756213, win 320, options [nop,nop,TS val 2148293344 ecr 2869738757], length 17
00:00:00.0008992 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236012, win 320, options [nop,nop,TS val 2869738938 ecr 2148293344], length 0
00:00:00.000190 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236012:2819236029, ack 1521756213, win 320, options [nop,nop,TS val 2148293483 ecr 2869738938], length 17
00:00:00.0007780 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236029, win 320, options [nop,nop,TS val 2869739079 ecr 2148293483], length 0
00:00:00.0008044 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236029:2819236046, ack 1521756213, win 320, options [nop,nop,TS val 2148293624 ecr 2869739079], length 17
00:00:00.0004370 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236046, win 320, options [nop,nop,TS val 2869739186 ecr 2148293624], length 0
00:00:00.000862 IP 127.0.0.1.15960 > 127.0.0.1.15960: Flags [P.], seq 2819236046:2819236063, ack 1521756213, win 320, options [nop,nop,TS val 2148293764 ecr 2869739186], length 17
00:00:00.0002395 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236063, win 320, options [nop,nop,TS val 2869739375 ecr 2148293764], length 0
00:00:00.0008045 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236063:2819236080, ack 1521756213, win 320, options [nop,nop,TS val 2148293907 ecr 2869739375], length 17
00:00:00.0003648 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236080, win 320, options [nop,nop,TS val 2869739461 ecr 2148293907], length 0
00:00:00.0008074 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236080:2819236097, ack 1521756213, win 320, options [nop,nop,TS val 2148294042 ecr 2869739461], length 17
00:00:00.00037495 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236097, win 320, options [nop,nop,TS val 2869739681 ecr 2148294042], length 0
00:00:00.000857 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236097:2819236114, ack 1521756213, win 320, options [nop,nop,TS val 2148294183 ecr 2869739681], length 17
00:00:00.0008443 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236114, win 320, options [nop,nop,TS val 2869739738 ecr 2148294183], length 0
00:00:00.0008289 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236114:2819236131, ack 1521756213, win 320, options [nop,nop,TS val 2148294324 ecr 2869739738], length 17
00:00:00.00032022 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236131, win 320, options [nop,nop,TS val 2869739876 ecr 2148294324], length 0
00:00:00.0008039 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236131:2819236148, ack 1521756213, win 320, options [nop,nop,TS val 2148294472 ecr 2869739876], length 17
00:00:00.0003650 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236148, win 320, options [nop,nop,TS val 2869740026 ecr 2148294472], length 0
00:00:00.0008027 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236148:2819236165, ack 1521756213, win 320, options [nop,nop,TS val 2148294611 ecr 2869740026], length 17
00:00:00.00038505 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236165, win 320, options [nop,nop,TS val 2869740167 ecr 2148294611], length 0
00:00:00.0008041 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236165:2819236182, ack 1521756213, win 320, options [nop,nop,TS val 2148294755 ecr 2869740167], length 17
00:00:00.00037487 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236182, win 320, options [nop,nop,TS val 2869740311 ecr 2148294755], length 0
00:00:00.0008070 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 2819236182:2819236199, ack 1521756213, win 320, options [nop,nop,TS val 2148294899 ecr 2869740311], length 17
00:00:00.0007743 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236199, win 320, options [nop,nop,TS val 2869740825 ecr 2148294899], length 0
00:00:00.0008064 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236199:2819236216, ack 1521756213, win 320, options [nop,nop,TS val 2148295041 ecr 2869740825], length 17
00:00:00.0004977 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236216, win 320, options [nop,nop,TS val 2869740609 ecr 2148295041], length 0
00:00:00.0008040 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236216:2819236233, ack 1521756213, win 320, options [nop,nop,TS val 2148295183 ecr 2869740609], length 17
00:00:00.0003281 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236233, win 320, options [nop,nop,TS val 2869741186 ecr 2148295183], length 0
00:00:00.0008357 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236233:2819236250, ack 1521756213, win 320, options [nop,nop,TS val 2148295319 ecr 2869741186], length 17
00:00:00.00045021 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236250, win 320, options [nop,nop,TS val 2869740883 ecr 2148295319], length 0
00:00:00.0008028 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236250:2819236267, ack 1521756213, win 320, options [nop,nop,TS val 2148295462 ecr 2869740883], length 17
00:00:00.0003761 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236267, win 320, options [nop,nop,TS val 2869741018 ecr 2148295462], length 0
00:00:00.0008040 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236267:2819236284, ack 1521756213, win 320, options [nop,nop,TS val 2148295608 ecr 2869741018], length 17
00:00:00.0003385 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236284, win 320, options [nop,nop,TS val 2869741160 ecr 2148295608], length 0
00:00:00.0008035 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236284:2819236301, ack 1521756213, win 320, options [nop,nop,TS val 2148295749 ecr 2869741160], length 17
00:00:00.0004690 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236301, win 320, options [nop,nop,TS val 2869741371 ecr 2148295749], length 0
00:00:00.0008042 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 2819236301:2819236318, ack 1521756213, win 320, options [nop,nop,TS val 2148295893 ecr 2869741371], length 17
00:00:00.00038146 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236318, win 320, options [nop,nop,TS val 2869741448 ecr 2148295893], length 0
00:00:00.0003736 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 1521756215, ack 2819236318, win 320, options [nop,nop,TS val 2869741552 ecr 2148295893], length 0
00:00:00.0008021 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [.], ack 1521756214, win 320, options [nop,nop,TS val 2148296036 ecr 2869741552], length 0
00:00:00.0008884 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [F.], seq 2819236318, ack 1521756214, win 320, options [nop,nop,TS val 2148296036 ecr 2869741552], length 0
00:00:00.0008012 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 2819236319, win 320, options [nop,nop,TS val 2869741552 ecr 2148296036], length 0
00:00:00.0008043 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 794015942:794015960, ack 2826931961, win 320, options [nop,nop,TS val 22855181 ecr 608618724], length 18
00:00:00.0008086 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [.], ack 794015960, win 320, options [nop,nop,TS val 608618724 ecr 22855181], length 0
00:00:00.0005383 IP 127.0.0.1.15960 > 127.0.0.1.5000: Flags [P.], seq 2826931961:2826931976, ack 794015960, win 320, options [nop,nop,TS val 608618813 ecr 22855181], length 15
00:00:00.000497 IP 127.0.0.1.5000 > 127.0.0.1.15960: Flags [P.], seq 794015968:794015977, ack 2826931976, win 320, options [nop,nop,TS val 22855270 ecr 608618813], length 19
[1] Done tcpdump -i lo0 -n -s0 -w exp8.pcap tcp port 5000
root@freedsd:~/col334_a2 #

```

Figure 8b: Packet capture confirming kernel-issued FIN handshake following SIGKILL.

Wire Event Chronology:

15960 > 5000: Flags [.]	ack 1	(Last subscriber ACK)
15960 > 5000: Flags [F.]	seq 1 ack 1	(Kernel-generated FIN on SIGKILL)
5000 > 15960: Flags [.]	ack 2	(Server ACK)
5000 > 15960: Flags [F.]	seq 1 ack 2	(Server FIN teardown)
15960 > 5000: Flags [.]	ack 2	(Final ACK, teardown complete)

Table 10: Process Termination vs. Order Survival Event Log

Time (ms)	Component	Action / Observation	Architectural Significance
3933.26	Kernel / TCP	SIGKILL sent to client; kernel emits FIN	Process death surfaces as normal EOF
3933.49	Server Reactor	eof_fin detected; close() triggered	Descriptor closed cleanly, removed from by_sid
3934.05	Server Reactor	Descriptor destroyed (nconn=3)	Surviving connections continue uninterrupted
4012.00	Trading Engine	Subsequent BUY/SELL pairs submitted	Matching engine executes subsequent trades normally
4012.10	Market Data	Surviving subscriber receives TRADE updates	Broadcast pipeline remains fully functional

9.4 Supplementary Proof: Resting Order Survival Post-Disconnect

Because the harness in Experiment 8 terminates a Market-Data subscriber (which holds no orders), a dedicated test verified the survival of resting trader orders:

1. Trader A connects (`sid=2`), places `BUY JNST 100 238` (order 0), and disconnects.
2. Trader B connects (`sid=3`), places `SELL JNST 60 238`.
3. Order 0 matches for 60 units. Trader B receives `SOLD JNST 60 238`, active subscribers receive `TRADE JNST 60 238`, and Trader A's private notification is safely dropped:

```
close sid=2 why='FIN' role='trader' orders_surviving=1
engine trade match: BUY 0 vs SELL 1 -> 60 JNST @ 238
notify_dropped: target_sid=2 msg=b'BOUGHT JNST 60 238\n'
```

10. System Concurrency & Protocol Verification

10.1 Handout §4.3 Concurrency Verification (≥ 10 Clients)

Handout Section 4.3 mandates simultaneous support for at least 10 active clients, including at least 2 Trader Clients and at least 4 Market-Data Clients.

This requirement was validated using `src/tools/gapd_concurrency.sh`, which launches 4 concurrent Trader Clients and 6 concurrent Market-Data Clients (10 total) against an active server. All 10 connections established and operated simultaneously without degradation:

```
$ sockstat -4 | grep :5000 | grep ESTABLISHED | wc -l
20    # 10 client endpoints + 10 server-side peer endpoints
```

10.2 Outbound and Inbound Message Coalescing

TCP stream framing was verified in both directions:

- **Inbound Coalescing:** Multiple complete commands transmitted in a single TCP segment (`LOGIN client_1\nBUY JNST 100 238\n`) are sequentially parsed and dispatched by `Framer.feed()`.
- **Outbound Latency Optimization:** `queue_out()` issues opportunistic flushes with `TCP_NODELAY`. Under normal conditions, replies are dispatched immediately; when peer backpressure occurs, messages coalesce automatically in userspace `wbuf`, optimizing throughput.

10.3 Protocol Error Vocabulary & Errno Matrix

The server handles errors strictly and uniformly across all interaction paths:

Table 11: Operating System Errno Mapping

Errno	Symbol	System Event	Implementation Response
35	<code>EAGAIN / EWOULDBLOCK</code>	Socket send buffer full	Catch <code>BlockingIOError</code> , buffer in <code>wbuf</code> , arm <code>KQ_FILTER_WRITE</code> .
54	<code>ECONNRESET</code>	Peer abortive reset (RST)	Detected via <code>ev.fflags=54</code> , mark dead, purge session.
32	<code>EPIPE</code>	Write to closed peer	Intercepted via <code>BrokenPipeError</code> , close descriptor cleanly.

Errno	Symbol	System Event	Implementation Response
60	ETIMEDOUT	Peer dropped silently	Handled by OS TCP keepalive / retransmit exhaustion.

11. Codebase Structure & Reproduction Commands

11.1 Source Code Architecture

server/run-server	Executable launcher for Exchange Server
client/run-trader	Executable launcher for Trader Client
client/run-market-data	Executable launcher for Market-Data Client
src/server.py	Main kqueue reactor, buffer manager, and network I/O
src/engine.py	Order book, matching engine, and session management
src/protocol.py	Byte-level parsing, numerical validation, and error strings
src/framing.py	Stream framing and newline delimiter extraction
src/trader.py	Interactive/automated trader client (multiplexed stdin)
src/market_data.py	Interactive/automated market data client
tests/test_all.py	Offline verification suite (framing, engine, validation)

11.2 Reproduction Commands

To run the automated experiment suite:

```
# Ensure launchers have execute permissions
chmod +x server/run-server client/run-trader client/run-market-data

# Execute experiments 1 through 8
python3 experiment.py 1
python3 experiment.py 2
python3 experiment.py 3
python3 experiment.py 4
python3 experiment.py 5
python3 experiment.py 6
python3 experiment.py 7
python3 experiment.py 8

# Run the 10-client concurrency test (§4.3)
sh src/tools/gapd_concurrency.sh 127.0.0.1 5000
```

12. Edge-Case Verification

Additional terminal-based checks were performed against the framing, protocol, matching, session-management, and socket write paths. These checks supplemented the numbered experiments and did not require any changes to the implementation.

- **TCP framing and incomplete input:** Commands split across multiple reads were reconstructed correctly, while several commands in one read were processed individually. A connection ending midway through a command did not execute the incomplete command.

- **Session and role handling:** Duplicate usernames were rejected, usernames became available again after disconnection, unauthenticated trading commands were rejected, and Market-Data clients could not place orders.
- **Numeric validation:** Zero, negative, malformed, and out-of-range quantities, prices, and order identifiers produced the corresponding protocol errors instead of entering the matching engine.
- **Cancellation behavior:** Cancellation was verified for partially filled orders, repeated cancellation, unknown order identifiers, fully executed orders, and attempts made by a trader who did not own the order.
- **Matching behavior:** Partial fills preserved the correct remaining quantity, one incoming order could fill against multiple resting orders, and equal-price orders were matched in arrival order.
- **Disconnection cleanup:** Outstanding orders remained available for matching after their trader disconnected. Private notifications to departed traders were dropped safely, subscribed clients still received trade updates, and disconnected Market-Data clients were removed from later broadcasts.
- **Backpressure and graceful termination:** A non-reading peer accumulated buffered output without blocking service to other clients. Once it resumed reading, the backlog drained. On QUIT, queued replies were sent before the connection closed, and commands following QUIT in the same batch were ignored.

All checks produced the expected protocol state and output.